Tic-Tac-Toe Game Documentation

Table of Contents

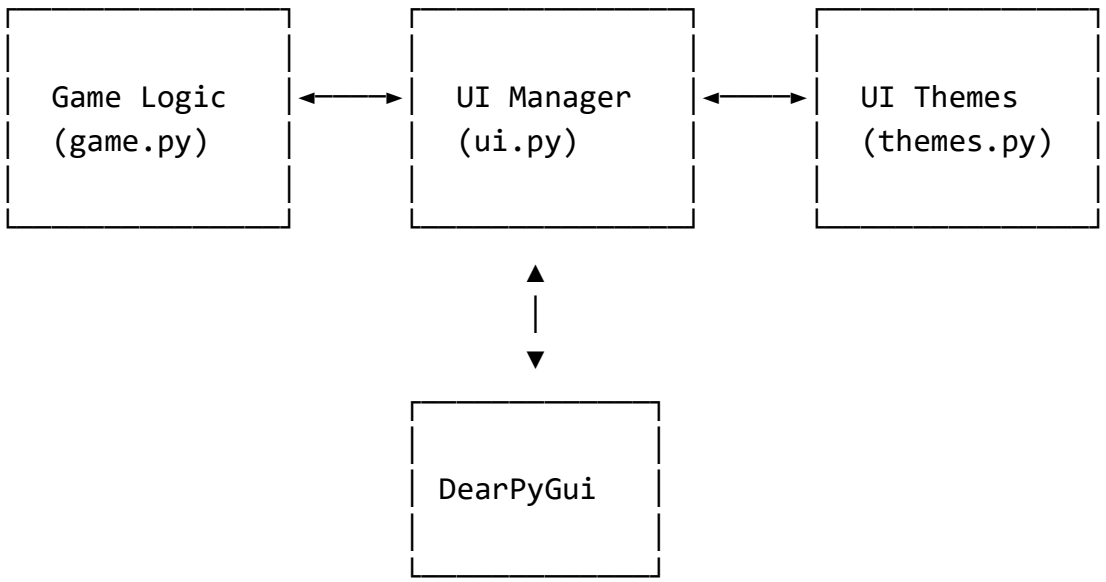
1. Introduction
2. System Architecture
3. Module Descriptions
    a. Game Logic Module
    b. UI Module
    c. Themes Module
4. Class Reference
    a. TicTacToeGame
    b. TicTacToeUI
5. AI Implementation
6. User Guide
7. Development Guide
8. Testing

Introduction

This Tic-Tac-Toe application is a modern implementation of the classic game with a graphical user interface. It supports both single-player mode against AI opponents of varying difficulty and a two-player mode for in-person play. The game offers a sleek, responsive interface with visual feedback and score tracking.

System Architecture

The application uses a modular architecture that separates the game logic from the user interface:



- **Game Logic Module**: Manages the game state, rules, and AI logic
- **UI Module**: Handles user interactions and display
- **Themes Module**: Defines the visual styling of the application
- **DearPyGui**: External library used for the graphical interface

Module Descriptions

Game Logic Module

The `game.py` module contains the `TicTacToeGame` class, which is responsible for:

- Maintaining the game state (board, current player, etc.)
- Validating moves
- Determining when a player has won or the game has ended in a tie
- Implementing the AI opponent using the minimax algorithm
- Tracking scores

UI Module

The `ui.py` module contains the `TicTacToeUI` class, which handles:

- Creating and managing the graphical interface
- Handling user input
- Reflecting the game state in the UI
- Managing animations and visual feedback
- Providing game controls (reset, difficulty selection, etc.)

Themes Module

The `themes.py` module defines the visual appearance of the game, including:

- Color schemes for different game elements
- Button styles for different states (X, O, winning move)
- Global styling for the application window
- Typography settings

Class Reference

TicTacToeGame

The main class for game logic.

Properties

Property	Type	Description
tabla	list	9-element list representing the 3x3 game board
jelenlegiJatekos	str	Current player ('X' or 'O')
gameOver	bool	Indicates if the game has ended
nyertes	str	Winner ('X', 'O', or 'Tie') or None if game in progress
pontszam	dict	Score tracking for 'X', 'O', and 'Tie'
gameActive	bool	Controls game interaction state
nehezseg	str	AI difficulty ('easy', 'medium', 'hard')
jatekMod	str	Game mode ('ai' or 'two_player')
winningCombo	list	Positions of winning combination when game is won

Methods

Method	Parameter	Return	Description
make_move	position	None	Attempts to place a mark at the given position
check_nyertes	None	bool	Checks if the game has been won or tied
reset_game	None	None	Resets the game state to start a new game
get_ai_move	None	int	Determines the AI's next move based on difficulty
best_move	None	int	Uses minimax to find optimal move (for hard AI)
minimax	depth, is_maximizing	int	Minimax algorithm implementation for AI
random_move	None	int	Selects a random valid move (for easy AI)
ai_move	None	None	Executes the AI's chosen move

TicTacToeUI

The main class for the user interface.

Properties

Property	Type	Description
game	TicTacToeGame	Reference to the game logic instance
themes	dict	Collection of UI theme references

Methods

Method	Parameter	Return	Description
setup_dpg	None	None	Initializes the DearPyGui interface
handle_button_click	position	None	Handles game board button clicks
handle_reset_game	None	None	Resets the game when reset button is clicked
set_jatekMod	value	None	Updates game mode based on radio button selection
set_nehezseg	value	None	Updates AI difficulty based on radio button selection
update_ui	None	None	Updates the UI to reflect current game state
run	None	None	Starts the game loop

AI Implementation

The AI opponent uses the minimax algorithm to determine optimal moves in the "Hard" difficulty setting. The algorithm evaluates possible future game states to select the most advantageous move.

Difficulty Levels

- **Easy**: Makes completely random moves
- **Medium**: Uses the minimax algorithm 70% of the time, and random moves 30% of the time
- **Hard**: Always uses the minimax algorithm to find the optimal move

Minimax Implementation

The minimax algorithm recursively evaluates all possible game states to a certain depth:

1. It simulates making a move
2. Then it evaluates the resulting board state
3. Then it simulates the opponent's best response
4. And so on until it reaches a terminal state (win, loss, or tie)

The algorithm assigns a score to each terminal state (+10 for AI win, -10 for player win, 0 for tie), and these scores are propagated back up the game tree to determine the best move for the current state.

User Guide

Game Settings

1. **Game Mode Selection**:
    a. Select "Against AI" to play against the computer
    b. Select "Two Players" to play with a friend locally
2. **AI Difficulty** (when playing against AI):
    a. Easy: Suitable for beginners
    b. Medium: Moderate challenge
    c. Hard: Very challenging, makes optimal moves

Gameplay

1. Click on an empty square to place your mark
2. X always goes first, followed by O
3. The first player to get three in a row (horizontally, vertically, or diagonally) wins
4. If all squares are filled with no winner, the game ends in a tie
5. Click "New Game" to start a new game while preserving scores

Visual Indicators

- Blue buttons: X player's moves
- Red buttons: O player's moves
- Green buttons: Winning combination
- Status text: Shows current player or game outcome
- Score tracking: Shows number of wins for each player and ties

Development Guide

Adding New Features

To add new features to the game:

1. **Game Logic Changes**:
    a. Modify `TicTacToeGame` class in `game.py`
    b. Add new game state properties or methods as needed
    c. Update existing methods that would be affected
2. **UI Changes**:
    a. Add new UI elements to the DearPyGui setup in `ui.py`
    b. Create callback functions for new controls
    c. Update the `update_ui` method to reflect new game state elements
3. **Styling Changes**:
    a. Add new themes to `setup_themes` function in `themes.py`
    b. Make sure to add new theme tags to the returned themes dictionary

Project Structure

When adding new modules or functionality:

1. Place core game components in the `TicTacToe` package
2. Add tests in the `tests` directory, mirroring the structure of the source code
3. Update `setup.py` and `requirements.txt` if adding dependencies

Testing

The project includes comprehensive unit tests for both game logic and UI components.

Game Logic Tests

Located in `tests/test_game.py`, these tests verify:

- Game initialization
- Move validation
- Win detection in various scenarios
- Game reset functionality
- AI move generation

UI Tests

Located in `tests/test_ui.py`, these tests verify:

- UI initialization
- Button click handling
- Game mode and difficulty setting
- UI updates for different game states

Running Tests

Execute tests using pytest:

```
python -m pytest
```

For more detailed output:

```
python -m pytest -v
```